Innovations in Systems and Software Engineering
https://doi.org/10.1007/s11334-025-00602-8

S.I.: MODELS/MODEVVA'22 & SAM'22

Visual animation of B specifications using executable DSLs

Asfand Yar[1] · **Akram Idani**[1] · **Yves Ledru**[1] · **Simon Collart-Dutilleul**[2]

Received: 10 February 2023 / Accepted: 27 March 2025

Abstract
Visual animation of formal specifications is useful for validation because it facilitates showing visually that the specifications satisfy the user's perception of requirements. The technique is especially useful for domain experts who would not be expected to understand formal specifications. However, in most tools, the development of a visual animation is done by formal methods engineers and requires skills in various technologies (e.g. Flash, JavaScript, SVG). Our work contributes toward the tools that are dedicated to the B method, such as B-Motion Studio, VisB, etc. In this paper, we show how visual animation can be done using a domain-specific language (DSL), which is expected to be used by domain experts. The advantage is that the mapping between the DSL and the formal specification is written in B itself. The proposed approach is supported by Meeduse, a language workbench built on ProB, an animator and model-checker of the B method. This paper also explains the DSL tool where the mappings between the DSL and formal specification are generated in a semi-automated way.

Keywords B method · Animation · MDE · DSLs

Abbreviations

DSL	Domain specific language
EMF	Eclipse modeling framework
ERTMS/ETCS	European rail traffic management system European train control system
MDE	Model-driven engineering
SVG	Scalable vector graphics
TTD	Trackside train detection

1 Introduction

Validation of specifications is essential because misunderstandings of user requirements can lead to erroneous implementations. In fact, specification faults may originate from human errors when reading and analyzing requirements. Validation in the context of formal specifications is known to be complex due to mathematical notations [1, 2]. M.-C. Gaudel in [2] analyzes specification faults and states that "a formal specification can be wrong for two reasons: some misunderstanding of user needs, some errors in the expression of these needs." In this paper, we are concerned only with the first reason, as the second one is widely addressed by works from the Requirements Engineering field. In theory, one way to avoid misunderstanding of user needs is to build concise and simple formal models using abstraction [2]. In B [3], for example, this is expected to be achieved step by step using refinements. However, most case studies in formal methods, such as the one discussed later in this paper, result in specifications that are neither simple nor concise. Another major difficulty of validating formal models comes from the overhead of the underlying mathematical notations. In [4], the authors mention that the learning curve of formal methods is steep, whereas the learning curve for drawing diagrams on a blackboard is very low. This observation is true due to the

✉ Asfand Yar
asfand.yar.ugaredirect@gmail.com

Akram Idani
akram.idani@univ-grenoble-alpes.fr

Yves Ledru
yves.ledru@univ-grenoble-alpes.fr

Simon Collart-Dutilleul
simon.collart-dutilleul@univ-eiffel.fr

[1] CNRS, Grenoble INP, LIG, Univ. Grenoble Alpes, 38000 Grenoble, France

[2] Univ. Gustave Eiffel, Univ. de Lille, 59650 Villeneuve d'Ascq, France

Published online: 03 June 2025

complexity of the mathematical notations that support formal methods.

To address validation, several formal tools [5–7] provide graphical animation techniques. The intention is to exhibit a visual representation of data and behaviors of a formal specification, making it more readable for domain experts. Visual animation is hence an interesting way to inspect whether formal specifications meet the user requirements. This work focuses on graphical animation and visualization of B specifications and provides a new approach complementing tools such as BRAMA [8], AnimB[1] B-Motion Studio [5], B-Motion Web [9], and VisB [10]. Existing tools have shown their strengths in practice, but still, some concerns remain, which motivates the current work. First, visual animation often requires specific skills such as in scripting languages (Flash,[2] JavaScript, node.js), or in Scalable Vector Graphics (SVG files), etc. These technologies are not necessarily mastered by the formal methods expert and can be cumbersome to learn and use. In [11], Krings and Körner observed that "When errors occur, it is not clear in which layer the cause is located: is it an error in the B model? Is an SVG file broken? Is the config file incorrect? Is there a bug in the JavaScript code? As some errors are not reported, development can be cumbersome if one is not an expert in all technologies." Second, mapping a specification to a domain representation is time-consuming and may be error-prone. Often this is done by a formal methods expert who is trying to document his own specification. Unfortunately, if the process of validation reveals some misunderstandings, not only must the formal specifications be re-worked, but also the visual representations and the underlying mapping. Furthermore, the formal methods expert might not be familiar with the domain-specific notations.

To address the aforementioned concerns, we propose a new technique built on domain-specific languages (DSLs). In our approach, the domain-specific representations are designed using a DSL tool created with EMF [12], a well-known MDE platform. The challenge is to bridge the gap between the DSL and the formal specification. To this purpose, we use Meeduse [13, 14], the only existing language workbench today that allows formal reasoning about the correctness of a DSL as well as the execution of the underlying formal semantics. Meeduse takes advantage of B specifications to define the semantics of a DSL and embeds ProB [15] for execution capabilities. Our approach favors the separation of concerns principle: the formal methods expert is responsible for the development of the DSL's semantics using B specifications, and the domain expert defines input models in the DSL syntax. Models provided by the domain expert are animated in Meeduse using ProB since their semantics are defined using B. Furthermore, our approach involves another actor, the MDE expert, who is responsible for developing the DSL tool.

This paper is an extension of [16], where we proposed an architecture to map domain-specific models and formal specifications using a linkage specification. The contribution of this paper is to introduce a DSL-based tool to automate the extraction of the linkage specification. The approach is built on defining generation patterns created by the user, which can be applied and reused for various specifications and models. To this purpose, we propose two DSLs: the first DSL defines generation patterns in a generic way, and the second DSL allows one to select patterns and describe in which order they should be applied. Our tool builds on these DSLs to produce the linkage specifications for a given model and specification.

The structure of this paper is as follows: Sect. 2 gives a brief overview of the B method. Section 3 explains the Meeduse language workbench. Section 4 presents and illustrates the main concepts of our approach via a simple example. Then, Sect. 5 demonstrates how a DSL can be used to ensure visual animation in Meeduse. Section 6 explains how our approach is automated and introduces our support tool. Applications to other examples are discussed in Sect. 7. Discussions about these applications are provided in Sect. 8. Section 9 reviews related work. Finally, Sect. 10 presents the conclusion and perspectives of this paper.

2 Overview of the B method

This section gives a brief overview of the B method; for more details about the B theory, we refer the reader to the B-Book [3]. Roughly speaking, B is a formal method based on mathematical foundations: set theory and first-order logic. It covers both the structure and the behaviour of a system. The structure is characterized by a set of variables and constants (called *data*) and the behaviour is performed by a set of operations built on the generalized substitution theory.

2.1 Abstract machines

The notion of abstract machine is fundamental in B. It provides a structured development composed of three parts (Fig. 1): header, declarations, and operations.

1. The header part: introduces the name of the machine, its parameters, and the constraints of these parameters.
2. The static (declarative) part: includes sets, constants, and variables. It also lists the properties of the constants and a characterization of the machine state using invariants. The latter are expressed in first-order predicate logic.
3. The dynamic (behavioral) part: includes the initialization of the variables and the various operations. The task of an

[1] AnimB: https://wiki.event-b.org/index.php/AnimB,

[2] Flash is outdated and is no longer maintained.

MACHINE
- HEADER PART
  - Name of the machine
  - Parameters
  - Constraints
- STATIC PART (DECLARATIVE)
  - Sets
  - Constants
  - Properties of the constants
  - Variables (*state*)
  - Invariant (*properties of the state*)
- DYNAMIC PART (OPERATIONAL)
  - Initialization of the variables
  - Operations

END

Fig. 1 Structural features of abstract B machines

operation is to modify the state of the machine without violating its invariant properties.

The B method masters the complexity of the formal development process using structured specifications. There are two kinds of structures: refinement and composition. Refinement is used to transform an abstract model into a more concrete one, and composition makes it possible to construct an abstract machine from smaller, already defined and proved ones.

2.1.1 Refinements

Starting from a high-level abstract specification and following a refinement process until the production of an executable program, the B method is a well-established development process. It has the ability to cover all the stages of a software life cycle. Indeed, B formal documents (Fig. 2) may define several abstraction levels ranging from preliminary design to coding. The transition from one level to another is called refinement and is guided by proof obligations, whose purpose is to establish a correct relationship between the abstraction levels. This decomposition into successive proved refinements allows modular and safe developments.

In B, there are two kinds of refinements: behaviour refinement and data refinement. The behaviour refinement means that the operation body is changed by an other one but without violating neither the conditions under which the initial operation is defined nor its possible executions. Data refinement applies a new set of data in the refined model, that is some data can be replaced and some data can be added. When data are being replaced, a linkage invariant must be defined to specify a conformity relationship between the old data and the new ones.

2.1.2 Composition

The B method provides an assembly mechanism aiming at the creation of structured and interdependent components. The objective, as stated in [3], is to compose the specifications of two (or more) abstract machines and thus, eventually, construct abstract machines in an incremental way. The composition mechanism in B, establishes the rules under which an abstract machine can access (read, write) data and operations issued from other machines. There are five clauses that ensure this composition: INCLUDES, SEES, USES, EXTENDS, PROMOTES and IMPORTS.

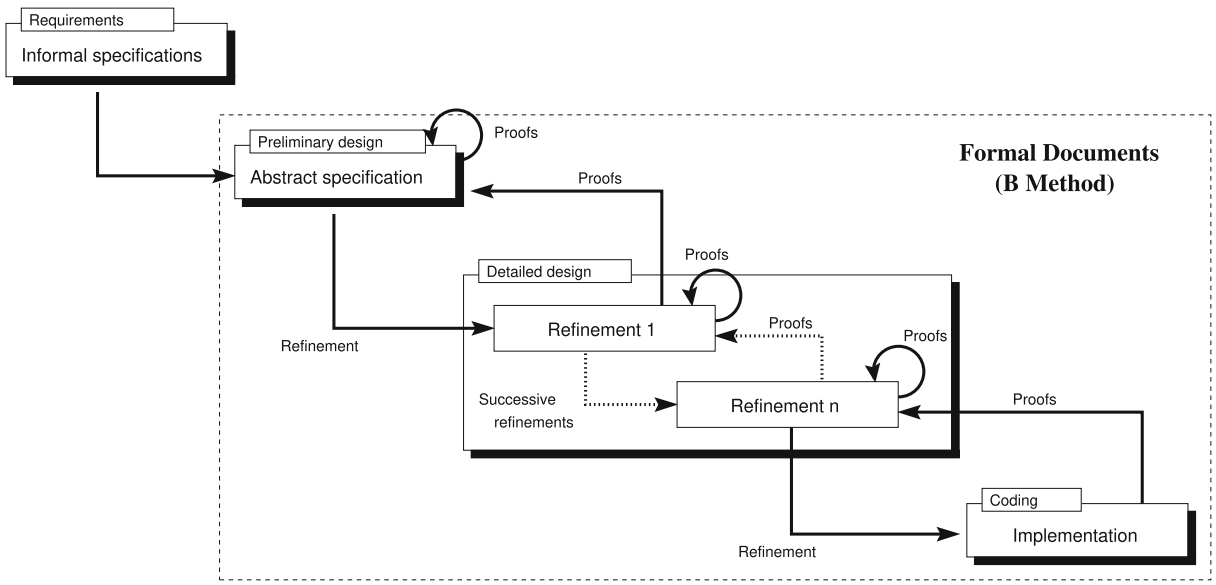


Fig. 2 Formal development process

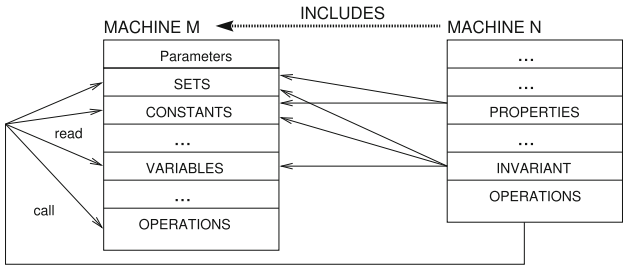


Fig. 3 Clause INCLUDES

The mostly used composition construct in the current document is the inclusion (clause INCLUDES). The latter is illustrated in Fig. 3 where two abstract machines M and N are presented, and such that N includes M. The visibility rules are therefore as follows: (*i*) the parameters of M are not accessible outside M, (*ii*) the variables of M can be read by N but their modification is only possible via operation call, (*iii*) the properties of N can refer to sets and constants from M, and (*iv*) the invariants of N can refer to sets, constants and also variables from M.

3 Meeduse

3.1 Main principles

Our domain-centric approach for visual animation builds on the Meeduse[3] language workbench [14]. The tool is dedicated to the formal instrumentation of domain-specific languages using the B-Method, which allows automated reasoning about their correctness. It embeds ProB [15] to ensure the animation and the verification of domain models. Figure 4 shows the main principles of the tool; for more details, we refer the reader to [13, 14].

Roughly speaking, the Meeduse approach addresses two layers: the semantics layer (top side of Fig. 4) and the modeling layer (bottom side of Fig. 4). In the semantics layer, the meta-model of a DSL is translated into B using a classical UML-to-B approach. This step leads to a functional B machine defining the static semantics of the DSL. Regarding the modeling layer, the Meeduse approach follows two steps − inject and animate − and benefits from ProB tool. Starting from a given model resource conforming to the DSL meta-model, Meeduse creates a valuated B machine that is semantically equivalent to the input model resource. This machine is an extraction of the functional B model in which Meeduse injects valuations to populate the various B data structures. Having this valuated machine, ProB is applied to animate B operations of any B machine that includes the valuated model. It is called "Dynamic Semantics" because it confers to the DSL a behavioural character. At every animation step, when ProB modifies the internal state of the valuated functional model, Meeduse translates back this modification to the input model resource, which results in an automatic animation of the model resource.

3.2 Semantics layer

To illustrate the Meeduse approach let us consider class `Counter` of Fig. 5. This class contains attribute `value` of type NAT.

Figure 6 shows the underlying B machine that is automatically extracted by Meeduse. For this example we asked the tool to generate variables only from class attributes. Hence, in the resulting B machine, attribute `value` is a variable whether class `Counter` is translated into a constant.

The tool also extracts basic operations, e.g. creation/deletion, setters and getters. Figures 7 and 8 show respectively the getter and the setter of attribute `value`. The B specification (Figures 6, 7 and 8) generated from the meta-model defines the so-called *Static Semantics* and is produced by component Translator of Fig. 4.

Based on the formal B specification of the static semantics, the DSL designer can introduce, using B, the dynamic semantics of the DSL as well as underlying invariant properties. Machine `DynamicSem` of Fig. 9 is a simple example of a B specification describing dynamic aspects of the counter. In this machine we introduce invariant $ran(value) \subseteq 1..10$ and operation `inc`. The latter increases attribute value, but under the precondition $value(cc) < 10$ in order to preserve the invariant. Note that this operation calls the basic setter `setValue` presented in Fig. 7, thanks to the inclusion mechanism of B. By applying basic operations issued from the static semantics machine, operations of the dynamic semantics preserve the invariant properties generated from the meta-model.

In this sub-section we illustrated the semantics layer of the Meeduse architecture. First, a B machine is generated from the meta-model of a DSL, gathering B data structures and basic operations. Then, the DSL designer can formally define the behavioural aspects of the DSL using an additional machine that includes the previous one. This approach allows one to formally reason about the correctness of the DSL using provers and/or model-checkers supporting the B method.

3.3 Modeling layer and execution

The model of Fig. 10 is an instance of meta-class `Counter` represented in the syntax of UML object diagrams. In this model attribute `value` is equal to 5 (Fig. 11).

Having this instance, the Injector component of Meeduse creates refinement *MeeduseTuto_r* that refines machine *MeeduseTuto* to introduce the corresponding valuations. For this

[3] http://vasco.imag.fr/tools/meeduse/.

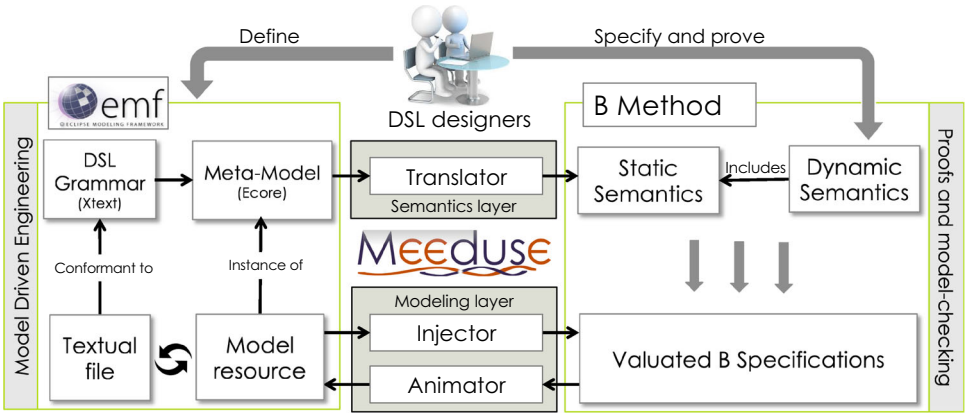


Fig. 4 Main principles of Meeduse (taken from [13])

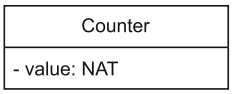


Fig. 5 A simple class

```
MACHINE MeeduseTuto
SETS
    COUNTER
CONCRETE_CONSTANTS
    Counter
PROPERTIES
    Counter ∈ Pow(COUNTER)
CONCRETE_VARIABLES
    value
INVARIANT
    value ∈ Counter → NAT
INITIALISATION
    value :∈ Counter → NAT
END
```

Fig. 6 Translation of a meta-model in Meeduse

```
OPERATIONS
setValue(aCounter, aValue) ==
    PRE
        aCounter ∈ Counter
        ∧ aValue ∈ NAT
    THEN
        value(aCounter) := aValue
    END
```

Fig. 7 A basic setter generated by Meeduse

```
OPERATIONS
aValue ← getValue(aCounter) ==
    PRE
        aCounter ∈ Counter
    THEN
        aValue := value(aCounter)
    END
```

Fig. 8 A basic getter generated by Meeduse

```
MACHINE DynamicSem
INCLUDES MeeduseTuto
INVARIANT
    ran(value) ⊆ 1..10
OPERATIONS
inc ==
    ANY cc WHERE
      cc ∈ Counter ∧ value(cc) < 10
    THEN
      setValue(cc, value(cc) +1)
    END
```

Fig. 9 Example of dynamic semantics written in B

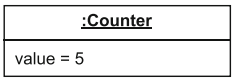


Fig. 10 A simple model

example, the unique instance of class `Counter`, leads to constant `COUNTER1`. Regarding relation `value`, it is initialized as {(*COUNTER1* ↦ 5)}.

Component Animator provides the valuated machine to ProB and then interactive animation starts. First, ProB animates the initialization and initiates the current state to *value* = { (*COUNTER1* ↦ 5) }. After this first state, the input model and ProB are synchronized. Then after every animation step Meeduse computes the difference between the state before and the state after, and updates the input model accordingly. For example, the state after the animation of operation `inc` is *value* = { (*COUNTER1* ↦ 6) }, which means that Meeduse updates the input model as presented in Fig. 12.

```
REFINEMENT MeeduseTuto_r
REFINES MeeduseTuto
CONSTANTS
    COUNTER1
PROPERTIES
    COUNTER1 ∈ COUNTER ∧
    Counter = {COUNTER1}
INITIALISATION
    value := {(COUNTER1 ↦ 5)}
END
```

Fig. 11 B model extracted by Meeduse

Fig. 12 An updated model

:Counter
value = 6

```
MACHINE
    Lift_existing
CONCRETE_CONSTANTS
    groundf, topf
PROPERTIES
    topf ∈ NAT ∧ groundf ∈ NAT
    ∧ (groundf < topf)
CONCRETE_VARIABLES
    call_buttons, cur_floor, direction_up,
    door_open
INVARIANT
    cur_floor ∈ (groundf .. topf)
    ∧ door_open ∈ BOOL
    ∧ call_buttons ∈ Pow(groundf .. topf)
    ∧ direction_up ∈ BOOL
    ∧ (door_open = TRUE ⇒
                cur_floor ∈ call_buttons)
INITIALISATION
    cur_floor := groundf
    || door_open := FALSE
    || call_buttons := ∅
    || direction_up := TRUE
```

Fig. 13 Lift example [17]

4 Overall approach

4.1 A simple example

Leuschel et al., in [17] used many examples for the visualization of B specifications. To illustrate our approach we select the Lift example which is well-known in formal verification and validation [18]. This specification has been already proved correct by its authors (e.g. the lift must not move while its doors are opened). The structural part of this specification, named $\text{Lift}_{\text{existing}}$, is given in Fig. 13.

It represents one lift that moves between the ground floor (constant groundf) and the top floor (constant topf). Both groundf and topf are natural numbers, such that groundf is less than topf. Variable cur_floor gives the current position of the lift among the floors. Variables direction_up and door_open are booleans, representing whether the direction of the lift is up and whether the door of the lift is open respectively. The call buttons are represented with variable call_buttons, a set of natural numbers, without any distinction between the internal and external buttons of the lift cabin. The specification contains seven B operations that are not all presented here for space reasons: move_up (presented in Fig. 14) move_down, reverse_lift_up, reverse_lift_down, open_door, close_door and push_call_button. We choose the lift specification because it is a pedagogical example used in the B-Book [3] and referenced in several papers [10, 17]. Furthermore, the example gathers several B data structures allowing us to illustrate various concepts of our approach.

```
move_up =
    SELECT
        door_open = FALSE
        ∧ cur_floor < topf
        ∧ direction_up = TRUE
        ∧ ∃ cc.((cc ∈ NAT )
        ∧ (cc > cur_floor)
        ∧ (cc ∈ call_buttons)))
        ∧ (cur_floor ∉ call_buttons)
    THEN
        cur_floor := ((cur_floor)+(1))
    END ;
```

Fig. 14 Operation move_up

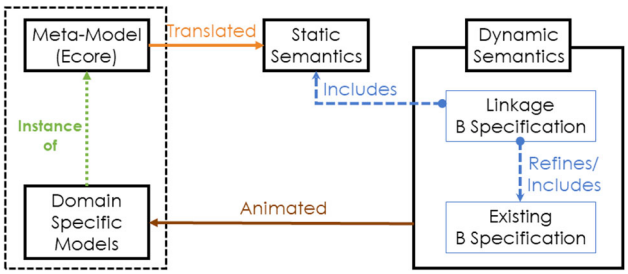


Fig. 15 Proposed approach

4.2 Proposed architecture

The B specification that defines the dynamic semantics of the DSL is often hand-written in Meeduse. It contains invariants and B operations that make the DSL executable. In this work, we propose an original use of the tool, where the specification corresponds to an existing B specification for which we want to provide visual animation. Our approach is illustrated in Fig. 15, which redefines the dynamic semantics part from Fig. 4.

We propose two techniques: Refinement/Inclusion and Inclusion/Inclusion. In the Refinement/Inclusion technique,

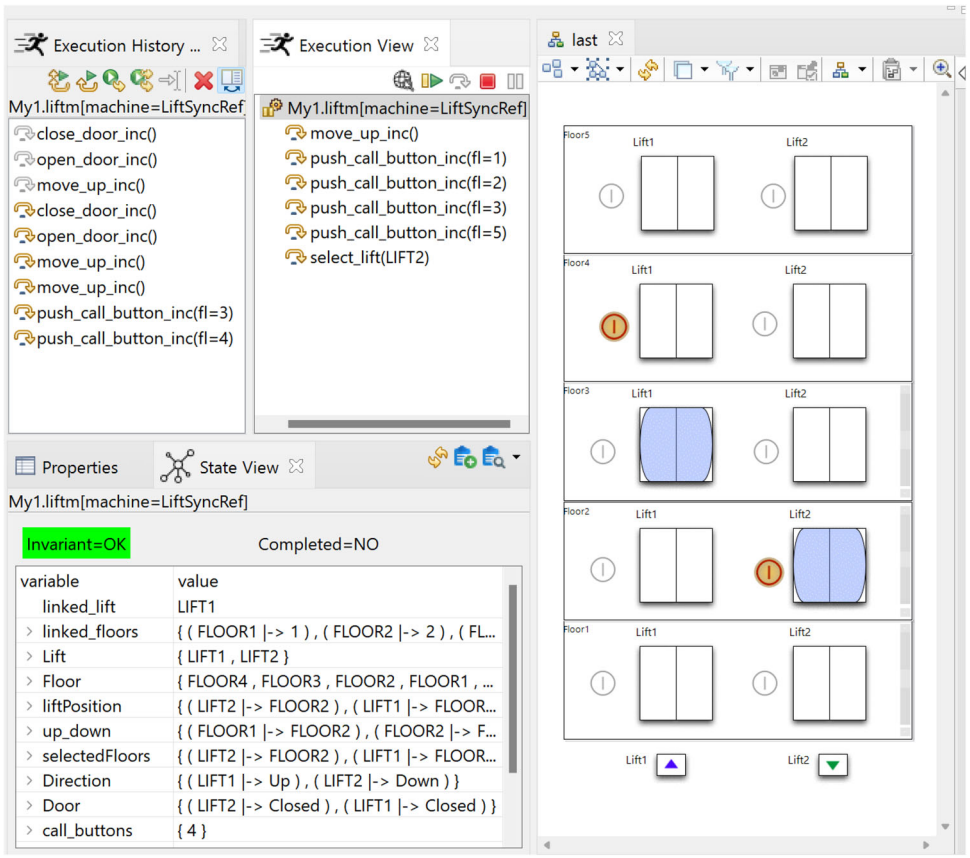


Fig. 16 Visual animation in Meeduse

B_{linkage} refines B_{existing} and includes B_{static}. In the Inclusion/Inclusion technique, B_{linkage} includes both B_{existing} and B_{static}.

Note that B_{linkage} applies B invariants to map the variables of B_{static} to those issued from B_{existing}. By animating B_{linkage}, the states of both B_{existing} and B_{static} are modified, and ProB verifies that the mapping is preserved throughout the animation. The task of Meeduse is to update the input model resource during the animation, conforming to the state of B_{static}, which produces the expected visual animation.

4.3 Illustration

Figure 16 shows a screenshot of Meeduse while animating an EMF-based DSL that represents the Lift example. The domain representation on the right-hand side of the figure can be realized by a domain expert using the modeling tool. The latter allows one to define buildings with several lifts. In this illustration, the model editor is developed with Sirius [19], an EMF-based framework dedicated to the design of graphical syntaxes of DSLs. The other views are for interactive animation. The execution view shows the candidate operations computed by ProB from B_{linkage}, and the state view shows the current valuations of the various B variables. The execution history, from the bottom, records the operation calls that led to the current state. After every animation step, the model resource is updated by Meeduse and Sirius automatically renders the graphical model without the need for any specific implementation effort.

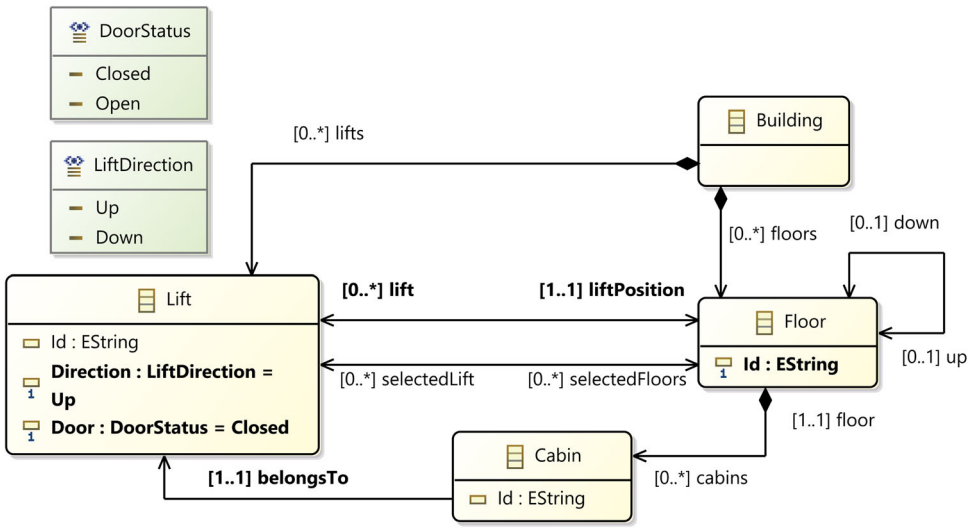
5 DSL-based visual animation

The previous section presented and illustrated a quick overview of the main concepts of our approach. In this section, we show, using the Lift example, how a DSL can be used to ensure visual animation in Meeduse.

5.1 The Lift DSL

Figure 17 gives the meta-model of our Lift DSL. The three main concepts of the DSL are buildings (class Building), floors (class Floor) and lifts (class Lift). Each floor has at most one upper floor (association up) and one lower floor (association down). Association liftPosition gives the position of a given lift, and selectedFloors refers to the set of selected floors from a given lift. Each floor is composed of zero or many cabins (association cabins), by "cabin" we mean the interface for a lift at a given floor. The concept of cabin is useful for our DSL because it manages the graphical representation: the visual aspect of a cabin changes depending on the current position of the lift and the door state. Class Lift

Fig. 17 Lift Meta-Model



has an attribute Direction that can be either Up or Down. Finally, attribute Door, represents the door, which can be either Closed or Open. The model presented on the right side of Fig. 16 is an instance of this meta-model. It gathers two lifts (Lift1 and Lift2) and five floors (Floor1, Floor2, Floor3, Floor4, and Floor5). Each floor has two cabins, corresponding to the two lifts. The arrow symbols at the bottom of the model show the direction of the lift. The direction of Lift1 is up while the direction of the Lift2 is down. The doors of both lifts are closed. The active button beside each door shows the selected floor for a given lift.

Note that the definition of the DSL is done independently from the existing B specification that one would like to visualize. The objective is to focus on the domain concepts and their relationships and build a DSL modeler that is useful for domain experts. For example, there is no explicit notion of call buttons. This concept is somehow similar to the association selectedFloors from Lift to Floor. This is also the case of class Cabin. This concept is not defined in the formal specification of the Lift but used in this simplified definition of the DSL.

5.2 Static semantics

The extraction of B_{static} is done by Meeduse, resulting in a B machine that covers the structure of the meta-model and gives several basic operations such as constructors, destructors, getters, and setters. Fig. 18 gives the structural part of this machine (named $Lift_{static}$).

In the generated B specification, we can notice that there are no data structures generated for the Building and Cabin classes. These concepts are useful for the lift DSL but we do not require them in our B specification. In fact, Meeduse features an annotation mechanism that allows the user to select the concepts to be translated. We generated seven variables

MACHINE
$Lift_{static}$
SETS
$LiftDirection = \{Up, Down\}$;
$DoorStatus = \{Closed, Open\}$;
$LIFT$; $FLOOR$
VARIABLES
$Lift$,
$Floor$,
$liftPosition$,
up_down,
$selectedFloors$,
$Direction$,
$Door$
INVARIANT
$Lift \in \text{Pow}\ (LIFT) \wedge$
$Floor \in \text{Pow}\ (FLOOR) \wedge$
$liftPosition \in Lift \rightarrow Floor \wedge$
$up\_down \in Floor \rightarrowtail Floor \wedge$
$selectedFloors \in Lift \leftrightarrow Floor \wedge$
$Direction \in Lift \rightarrow LiftDirection \wedge$
$Door \in Lift \rightarrow DoorStatus$

Fig. 18 Structural part of machine $Lift_{static}$

together with their typing invariants: Lift, Floor, liftPosition, up_down, selectedFloors, Direction, and Door. The operations of $Lift_{static}$ are not included here due to the limitation of space.

5.3 Linking B data structures

Figures 13 and 18 gave respectively the existing machine ($Lift_{existing}$) and the static semantics of the lift DSL ($Lift_{static}$). In order to apply Meeduse for visual animation, one needs to create a third machine ($Lift_{linkage}$) that maps B data of both machines.

5.3.1 Mapping a class to a machine

Machine $\text{Lift}_{\text{existing}}$ contains invariants and B operations to move a lift between the floors safely without any error. In machine $\text{Lift}_{\text{static}}$ multiple objects of class Lift can be created (abstract set LIFT and variable Lift). In this case, we consider that $\text{Lift}_{\text{existing}}$ will control only one instance of class Lift. Thus, we introduce in machine $\text{Lift}_{\text{linkage}}$ variable *linked_lift* to select the lift object that will be controlled by the existing machine:

Listing 1: EClass to BMachine

```
VARIABLE
  linked_lift
INVARIANT
  linked_lift ∈ Lift
INITIALISATION
  linked_lift :∈ Lift
```

5.3.2 Mapping a class to a set

In $\text{Lift}_{\text{existing}}$ floors are represented with a set of natural numbers ($groundf\,..\,topf$), and in $\text{Lift}_{\text{static}}$ they are defined with a dedicated variable representing class Floor. The mapping is then done as follows:

Listing 2: EClass to BSet

```
VARIABLE
  linked_floors
INVARIANT
  linked_floors ∈ Floor ⟼ groundf .. topf
INITIALISATION
ANY mapFloors WHERE
  mapFloors ∈ Floor ⟼ groundf .. topf ∧
  ∀ (f1,f2).(f1 ∈ Floor ∧ f2 ∈ Floor
      ∧ (f1 ↦ f2) ∈ up_down
        ⇒ mapFloors(f2) = mapFloors(f1) + 1)
  THEN
    linked_floors := mapFloors
  END
```

Relation $linked\_floors$ is a total bijection meaning that every floor in the DSL must be linked to one value from ($groundf\,..\,topf$) and vice-versa. The initialisation is done such that if a floor f2 is associated to a floor f1 via relation up_down then the value of mapFloors(f2) is equal to mapFloors(f1) plus one.

5.3.3 Mapping a single-valued reference to a set element

In the $\text{Lift}_{\text{existing}}$; the current floor of the lift is defined with variable cur_floor, which is an element of set ($groundf\,..\,topf$). This concept is defined in the DSL with reference liftPosition whose source and target classes have been already mapped with the rules above. The source has been mapped to a machine and the target to a set. To map cur_floor to reference liftPosition, we introduce the following invariant:

Listing 3: Single-valued EReference to a Set Element

```
INVARIANT
  linked_floors(liftPosition(linked_lift)) = cur_floor
```

5.3.4 Mapping a multi-valued reference to a set

The variable *call_buttons* of machine $\text{Lift}_{\text{existing}}$ is the set of pressed buttons that allows one to select the floors to which the lift is to be moved. This concept is represented in the DSL via reference selectedFloors. This leads to the following invariant:

Listing 4: Multi-valued EReference to a Set

```
INVARIANT
  linked_floors[selectedFloors[{linked_lift}]] =
call_buttons
```

5.3.5 Mapping an enumeration to boolean values

In $\text{Lift}_{\text{existing}}$ the Boolean variable *door_open* defines the status of the lift door (false if the door is closed and true if it is open). This concept is represented in the DSL via attribute Door of class Lift, whose type is enumeration DoorStatus. The same case happens for attribute Direction and variable *direction_up*. Mapping these concepts to each other introduces additional invariants to $\text{Lift}_{\text{linkage}}$:

Listing 5: EnumType to Boolean Values

```
INVARIANT
  (Door(linked_lift) = Closed ⇔ door_open = FALSE)
  ∧ (Door(linked_lift) = Open ⇔ door_open = TRUE)
  ∧ (Direction(linked_lift) = Down
      ⇔ direction_up = FALSE)
  ∧ (Direction(linked_lift) = Up ⇔ direction_up = TRUE)
```

5.4 Initialization

Once the data structures of both machines have been mapped to each other, using additional variables and invariants, we need to think about the initialization and the operations in order to keep the conformance of both models all along the animation. Machine $\text{Lift}_{\text{linkage}}$ includes machine $\text{Lift}_{\text{static}}$, which allows it to use the basic operations provided by the latter to update the state of its variables. $\text{Lift}_{\text{linkage}}$ includes or refines (depending on the strategy chosen by the user) machine $\text{Lift}_{\text{existing}}$. In both cases, the constants of $\text{Lift}_{\text{existing}}$

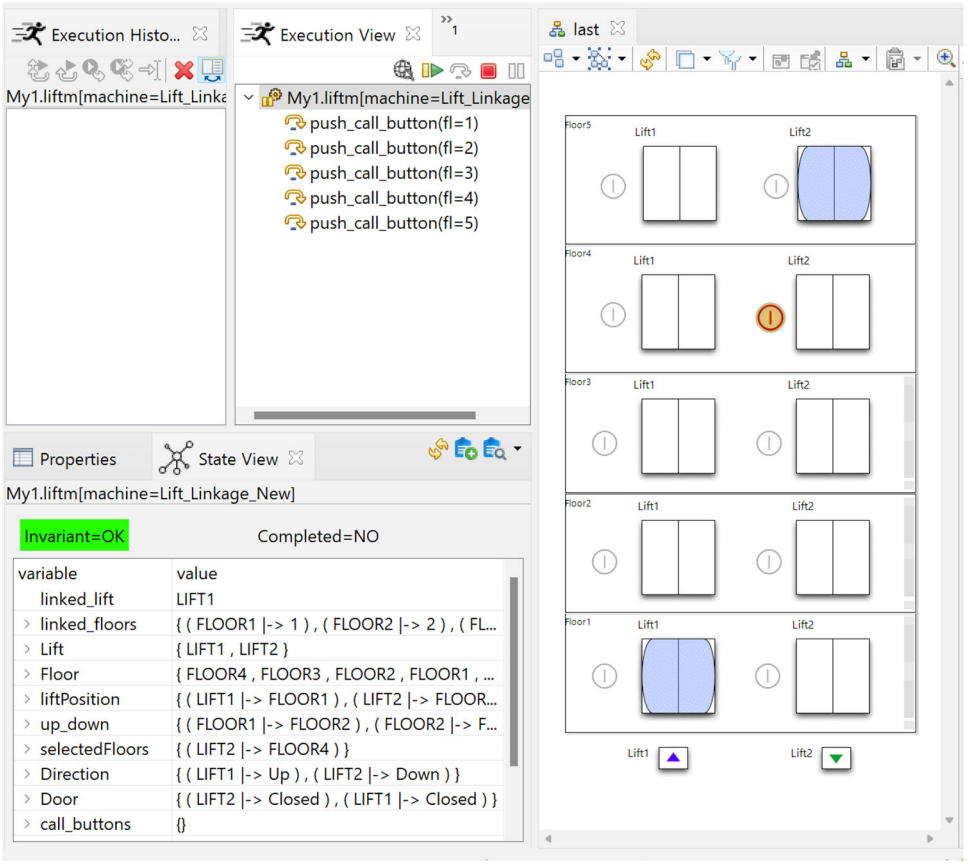


Fig. 19 Meeduse after the initialization

are visible and can be read by $\text{Lift}_{\text{linkage}}$. The initialization of the model is done as:

Listing 6: Initializing the Model

```
PROPERTIES
    card(groundf . . topf) = card(FLOOR)
INITIALISATION
    SetDirection(linked_lift,Up);
    SetDoor(linked_lift,Closed);
    UnsetSelectedFloors(linked_lift) ;
    SetLiftPosition(linked_lift,  linked_floors⁻¹(groundf))
```

Constants *groundf* and *topf* are not assigned to values in the existing specification. However, the visual animation starts from a model built in the DSL tool and in which buildings and floors have been already created. Thus, the PROPERTIES clause indicates to ProB to choose any valuation of these constants that respects the existing number of objects defined in the DSL model. Regarding the INITIALISATION clause, in addition to the initialisation of the linkage variables (Cf. listings 1 and 2), it updates the DSL model such that it starts in a state that is compliant with the initialisation of machine $\text{Lift}_{\text{existing}}$ given in Fig. 13. To this purpose we simply call basic operations provided by $\text{Lift}_{\text{static}}$ (i.e. `SetDirection`, `SetDoor`, `UnsetSelectedFloors` and `SetLiftPosition`). Figure 19 is a screenshot of Meeduse after the initialization.

The state view on the bottom left side of the figure shows that the linked_lift is LIFT1. Note that ProB gives the two possible initialisations of linked_lift (LIFT1 and LIFT2), and the user may select the one to be controlled with $\text{Lift}_{\text{existing}}$. In this case, we selected LIFT1. Besides, the DSL model deals with five floors, which are all mapped via linked_floors satisfying the underlying invariant. The liftPosition of LIFT1 is set to FLOOR1 because machine $\text{Lift}_{\text{existing}}$ starts with a lift that is at the ground floor.

5.5 Operations

We propose two strategies to build the dynamic semantics of the DSL by means of a linkage machine: (1) Refinement/Inclusion and (2) Inclusion/Inclusion. In both strategies, the static semantics machine is included in the linkage machine.

5.5.1 Refinement/Inclusion

In this strategy, the existing machine is refined by introducing the linkage variables and invariants, which suggests the refinement of the operations too. In Listing 7 below, we give the refinement of operation *move_up*. We copied, from

$\text{Lift}_{\text{existing}}$, the body of the operation (from line `2.` to line `10.`) and we introduced a call to operation *setLiftPosition* (line `11.`). This basic setter of relation `position` is provided by $\text{Lift}_{\text{static}}$; its task is to change the position of the lift conforming to the linkage. The objective is to preserve the invariant of Listing 3, since only variable *cur_floor* is modified by operation *move_up*.

Listing 7: Refining Operation *move_up*

```
OPERATIONS
1.  move_up =
2.   SELECT
3.    door_open = FALSE
4.     ∧ cur_floor < topf
5.     ∧ direction_up = TRUE
6.     ∧ ∃ cc.((cc ∈ Z ) ∧ (cc > cur_floor) ∧ (cc
       ∈ call_buttons))
8.     ∧ (cur_floor ∉ call_buttons)
9.   THEN
10.    cur_floor := ((cur_floor)+(1)) ;
11.  SetLiftPosition(linked_lift,linked_floors^−1
       (cur_floor))
12. END
```

5.5.2 Inclusion/Inclusion

In this strategy the existing machine is included in the linkage machine so that its operations can be used without a need to copy their body like in the Refinement/Inclusion strategy. The idea is to synchronise the states of both machines during the animation. For every operation of the existing machine, we create a synchronisation operation that manages the evolution of the two machines. For example, operation *move_up_inc* below calls both *move_up* and *SetLiftPosition*.

Listing 8: Synchronizing Models

```
OPERATIONS
move_up_inc =
 BEGIN
  move_up ;
  SetLiftPosition(linked_lift,          linked_floors
^−1(cur_floor))
 END
```

Both strategies have their advantages. Inclusion/Inclusion provides a lightweight approach to the usage of operations. It is convenient when the DSL or the existing machine is not yet finished and may be changed. The Refinement/Inclusion is better if one would like to continue the refinement process such that the variables from the existing machine are completely redefined using variables of the static semantics machine. This technique may be useful to produce step-by-step the dynamic semantics of a DSL from an existing machine that has been already proven correct. In this paper, we do not go further in this approach since our objective is to focus on how a DSL can ensure visual animation of formal B specifications.

5.6 Enhancements

The initialization provided in listing 6 initializes $\text{Lift}_{\text{static}}$ based on the initial state of $\text{Lift}_{\text{existing}}$. This approach is reasonable as far as the objective is visual animation − since we do not introduce any modification to the existing specification, we just visualize it. Nevertheless, the limitation is that every time we switch from one lift to another (e.g. from LIFT1 to LIFT2) we must animate the initialization with a different lift. The selected new lift is, therefore, re-initialized, which brings it back to the ground floor. A better solution would be to use $\text{Lift}_{\text{existing}}$ as a controller which can switch between the two lifts. To this purpose, we need to initialize $\text{Lift}_{\text{existing}}$ from $\text{Lift}_{\text{static}}$ and introduce an operation that allows one to select on-the-fly any lift without re-initializing it. The setter (setValues) of listing 9 is introduced within $\text{Lift}_{\text{existing}}$ in order to update its variables conforming to its invariants. This kind of setter can be generated automatically: every parameter is assigned to a variable and the precondition applies the invariants to the parameters.

Listing 9: Adding a Setter to $\text{B}_{\text{existing}}$

```
OPERATIONS
setValues(call_buttons_,cur_floor_,
direction_up_,door_open_) ==
 PRE
  cur_floor_ ∈ groundf. .topf
   ∧ door_open_ ∈ BOOL
   ∧ call_buttons_ ∈ Pow (groundf . . topf)
   ∧ direction_up_ ∈ BOOL
   ∧ (door_open_ = TRUE ⟹ cur_floor_ ∈
call_buttons_)
 THEN
  cur_floor := cur_floor_
   || door_open := door_open_
   || call_buttons := call_buttons_
   || direction_up := direction_up_
 END
```

To initialize $\text{Lift}_{\text{existing}}$ from $\text{Lift}_{\text{static}}$ we define the substitution of listing 10 and we use it in the initialization clause of $\text{Lift}_{\text{linkage}}$. In this case, we are applying the Inclusion/Inclusion approach. Definition update_existing calls setter setValues based on the linkage invariants. Once the lift and the floors are mapped, this definition finds a valuation to the

variables of $\text{Lift}_{\text{existing}}$ making them compatible with the valuations of $\text{Lift}_{\text{static}}$.

Listing 10: Initialize $\text{B}_{\text{existing}}$ from B_{static}

```
DEFINITIONS
update_existing ==
 ANY cur_floor_, door_open_, direction_up_, call_buttons_
 WHERE
  cur_floor_ = linked_floors(liftPosition(linked_lift))
  ∧ (Door(linked_lift) = Closed ⟹ door_open_ = FALSE)
  ∧ (Door(linked_lift) = Open ⟹ door_open_ = TRUE)
  ∧ (Direction(linked_lift) = Down
       ⟹ direction_up_ = FALSE)
  ∧ (Direction(linked_lift) = Up
       ⟹ direction_up_ = TRUE)
  ∧ linked_floors[selectedFloors[{linked_lift}]]
      = call_buttons_
 THEN
  setValues
  (call_buttons_, cur_floor_,direction_up_,
  door_open_)
 END
```

Finally, in order to switch between lifts at any time during the animation, we introduce the following operation select_lift. It selects a different lift, updates variable *linked_lift* and sequentially applies substitution update_existing. An illustration of this operation is provided in Fig. 16. By running operation select_lift(LIFT2) one can control LIFT2 without bringing it back to the ground floor, and later continue with LIFT1. Note that operation select_lift could apply a scheduling approach and provide an optimized technique when switching between lifts.

Listing 11: Change Mapping On-The-Fly

```
OPERATIONS
select_lift =
    ANY lift WHERE
        lift ∈ Lift ∧ lift ≠ linked_lift
    THEN
        linked_lift := lift ; update_existing
    END
```

6 A pattern-based approach to the extraction of linkage specifications

In order to generate the linkage machines, we first propose a textual DSL called Pattern-Definition that allows users to define patterns of mappings. Secondly, to apply the patterns defined with Pattern-Definition, we propose another DSL called Pattern-Application. Figure 20 shows the underlying approach. Pattern-Definition and Pattern-Application are textual DSLs conforming to their corresponding XText grammars. Using these DSLs, the user first proposes a catalog of generic reusable patterns and then selects the ones to apply. With the definition of patterns and their application, the component Linkage Generator automatically produces the linkage machine. This component is built using Acceleo [20], an Eclipse-based model-to-text transformation tool.

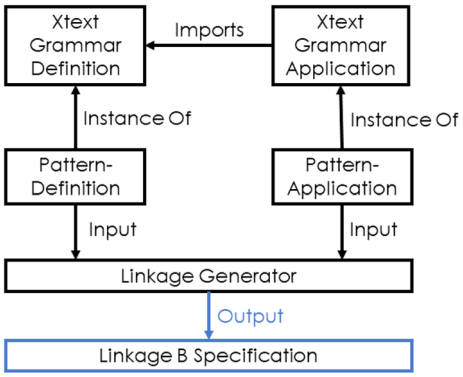


Fig. 20 Linkage Generation Methodology

6.1 Pattern-definition

Figure 21 gives the Xtext grammar of Pattern-Definition DSL, and Fig. 23 shows an instance of this grammar.

In this DSL, a template (defined with a rule Template) is a list of patterns (rule Pattern). The idea is that the user can define an exhaustive catalog of templates and patterns since there are numerous possible mappings depending on the application case. A pattern contains parameters, aliases, and mappings. Parameters are formal parameters that must be valuated in the application step, and aliases refer to local expressions that can be used in a mapping. Often parameters refer to a concept of a meta-model (*e.g.,* EClass) and the mapping indicates how this concept is mapped to concepts of a B machine. For example, in Fig. 23, the pattern EClassToB-Machine has only one parameter, which is an EClass, and produces a variable, an invariant, and an initialization. Rule Mapping has two parts: clause and alternative. It defines the result of the pattern and prescribes to which B clause this result must be added. An alternative is a mixture of aliases and free text.

Figure 22 provides the EMF meta-model of Pattern-Definition DSL defined from the given Xtext grammar shown in Fig. 21. The meta-model is composed of eight classes, each of which is issued from a grammar rule. Template is the root

```
Template:
    'template' name=ID
    (patterns+=Pattern)*;
Pattern:
    'pattern' name=ID '('
        (parameters+=Parameter)+
    ')'
    alias+=Alias*
    '{'
        (mapping+=Mapping)+
    '}'
;
Alias:
    'alias' name=ID ':' aliasAlt+=AliasAlternative+;
AliasAlternative :
    parameter=[Parameter] | text=STRING;
Mapping:
    clause=Clause ':' (alternative+=Alternative)+;
Clause:
    name=ID;

Alternative:
    alias=[Alias]  | text=STRING;
Parameter:
    name=ID;
```

Fig. 21 Xtext grammar of Pattern-Definition

```
template Lift
pattern EClassToBMachine(anEClass)
alias anEClass : anEClass
alias varName : "linked_"anEClass
{
    VARIABLES : varName
    INVARIANT : varName ":" anEClass
    INITIALISATION : varName "::" anEClass
}
pattern EClassToBSetExtended(anEClass aBSet userDefined)
alias userDefined : userDefined
alias varName : "linked_"anEClass
alias relation : anEClass ">->>" aBSet
{
    VARIABLES : varName
    INVARIANT : varName ":" relation
    INITIALISATION:
    "ANY mapping WHERE
        mapping :" relation "
    &
    "
        userDefined
    "THEN
    "
        varName ":= mapping
    END"
}
```

Fig. 23 User defined patterns

class and the associations represent the relationships between the objects.

Figure 23 defines a template (named Lift) with two patterns: EClassToBMachine and EClassToBSetExtended. Pattern EClassToBMachine is composed of three mappings with clauses: VARIABLES, INVARIANT and INITIALISATION respectively. In clause VARIABLES, this pattern adds a variable name, represented with alias varName whose value is the concatenation of string "linked_" with the value of parameter anEClass. The same principle is applied to the other clauses, (INVARIANT: varName ":" anEClass) and (INITIALISATION: varName "::" anEClass).

Pattern EClassToBSetExtended is defined with three parameters: anEClass, aBSet, and userDefined. Alias userDefined refers to the parameter userDefined while alias relation is defined as a bijection from parameter anEClass to parameter aBSet. Pattern EClassToBSetExtended is also composed of three mappings, that produce B concepts in clauses VARIABLES, INVARIANT and INITIALISATION.

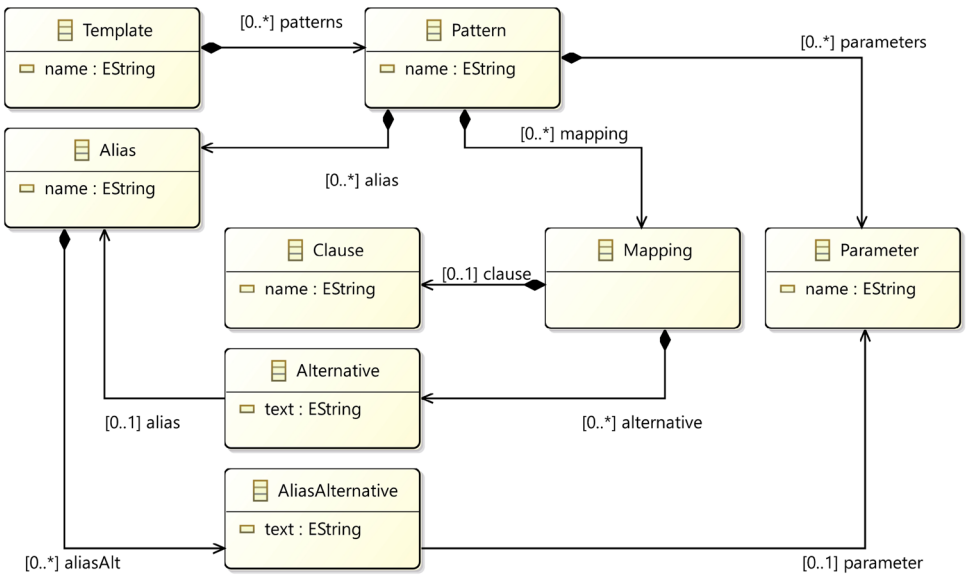


Fig. 22 Pattern-Definition meta-model

```
Application :
    'technique' technique=Technique
    (header=Header)
    ((applyPatterns+=ApplyPattern) | (bclauses+=BClauses))*;
BClauses:
    'BClause' name=ID
        bclause=OSTRING;
Header:
    'header' '{'
        headerfirst=HeaderFirst  name=ID
        text=OSTRING '}';
enum HeaderFirst:
    mach='MACHINE' | ref='REFINEMENT';
enum Technique:
    refinc= 'RefinementInclusion'
    | incinc= 'InclusionInclusion';
ApplyPattern:
    'applyPattern'
        pattern=[Pattern|QualifiedName]
            '(' value+=(ID | OSTRING)+ ')';
terminal OSTRING :
            'B{' -> '}B';
QualifiedName:
    ID ('.' ID)*;
```

Fig. 24 Xtext grammar of Pattern-Application

6.2 Pattern-application

The Xtext grammar of Pattern-Application DSL is given in Fig. 24 whereas Fig. 26 presents an instance of this grammar.

The objective of this DSL is to apply the patterns of a template defined using the Pattern-Definition DSL. Pattern-Application DSL is an application (Rule Application) with a technique (enum Technique), a header (Rule Header), multiples applyPatterns (Rule ApplyPattern), and bclauses (Rule BClauses). A technique can be one of our strategies, either Refinement/Inclusion or Inclusion/Inclusion. A header contains headerfirst (enum HeaderFirst), which is the type of B specification that can be either a machine or refinement. The header also contains the name of the B specification and a text part for the other part of the header like an OSTRING. We introduced OSTRING as a terminal rule to define the B syntax within B{ syntax }B. The applyPattern applies a pattern of a template and takes an exact number of parameters as defined using the Pattern-Definition DSL. The parts of the linkage specification, especially clauses that can not be produced by applying patterns, are defined manually in Bclauses, for instance, definitions, initialization, operations, etc.

The EMF meta-model of Pattern-Application is shown in Fig. 25. It is defined from the Xtext grammar shown in Fig. 24. The meta-model consists of four classes, where class Application is the root class. It is composed of three classes: BClauses, ApplyPattern, and Header. HeaderFirst and Technique are the two enumerations in the meta-model.

Figure 26 defines an application with technique Refinement/Inclusion. The type of B specification is defined as a refinement with the name Lift_Linkage, and the text part shows that it refines the Lift_Existing and includes the Lift_DSL. The application applies two patterns from the Lift template. The first pattern is the EClassToBMachine which takes anEclass Lift as a parameter. The second one is EClassToBSetExtended, which takes three parameters: anEClass, aBSet, and userDefined. We used the Floor as EClass, groundf..topf as the B set, and B syntax as user-defined. The application also contains a BClause which is an

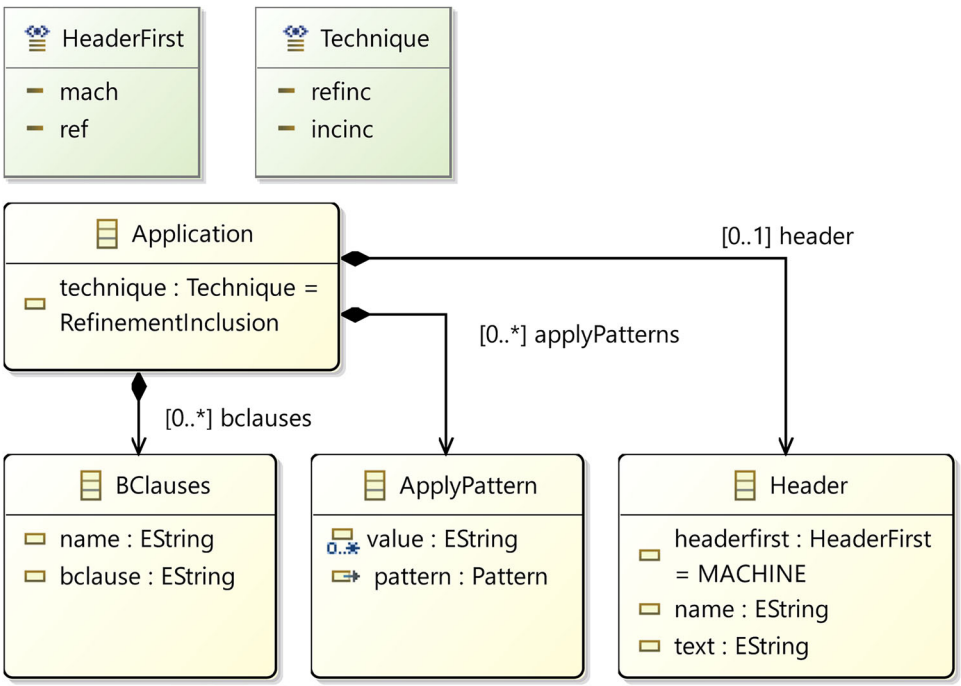


Fig. 25 Pattern-Application meta-model

```
technique RefinementInclusion
header{
    REFINEMENT Lift_Linkage
    B{
        REFINES Lift_Existing
        INCLUDES Lift_DSL
    }B
}
applyPattern Lift.EClassToBMachine(Lift)

applyPattern Lift.EClassToBSetExtended
    (
    Floor
    B{groundf..topf}B
    B{!(f1,f2).(f1 : Floor & f2 : Floor & (f1|->f2)
        : up_down => mapping(f2) = mapping(f1) + 1
    )}B
)

BClause INITIALISATION
B{   cur_floor := (groundf) ;
    door_open := FALSE ;
    call_buttons := ({}) ;
    direction_up := TRUE
}B
```

Fig. 26 Pattern-Application textual editor

```
pattern SingleValuedERefToSetElement
(EReferenceExp aSetElement)
alias EReferenceExp : EReferenceExp
alias aSetElement : aSetElement
{
INVARIANT : EReferenceExp "=" aSetElement
}
```

Fig. 27 SingleValuedERefToSetElement Pattern-Definition

initialisation where cur_floor, door_open, call_buttons, and direction_up are initialized.

6.3 Application to the lift example

We defined 5 definition patterns during the application of our tool to the example of Lift. To generate the Lift linkage B specification, we applied 6 application patterns and introduced 2 BClauses. The first two defined patterns of Lift example are already illustrated in Figs. 23 and 26. Following are the other three defined patterns and their application to the Lift example.

6.3.1 Single valued EReference to set element

As shown in Fig. 27, two parameters are defined for this pattern. The first one is EReferenceExp which is an expression for an EReference, and the other one is an element of a set. Two aliases are defined for local expressions. The pattern maps EReference with the set element and produces an invariant.

```
applyPattern Lift.SingleValuedERefToSetElement(
B{linked_floors(liftPosition(linked_lift))}B
cur_floor
)
```

Fig. 28 SingleValuedERefToSetElement Pattern-Application

```
pattern MultipleValuedERefToSet
(EReferenceExp aSet)
alias EReferenceExp : EReferenceExp
alias aSet : aSet
{
INVARIANT : EReferenceExp "=" aSet
}
```

Fig. 29 MultipleValuedERefToSet Pattern-Definition

```
applyPattern Lift.MultileValuedERefToSet(
B{linked_floors[selectedFloors[{linked_lift}]]}B
call_buttons
)
```

Fig. 30 MultipleValuedERefToSet Pattern-Application

Figure 28 shows the application of the pattern SingleValuedERefToSetElement where it takes two parameters as defined. EReferenceExp is the expression of liftPosition which is a single-valued reference from EClass linked_lift to EClass linked_floors. The cur_floor is the current position of the floor, and it is an element from the set groundf..topf.

6.3.2 Multiple valued EReference to set

Figure 29 illustrates the pattern MultipleValuedERefToSet. Two parameters are defined for this pattern: An EReferenceExp and aSet. The pattern also contains two aliases for local expressions. The pattern produces an invariant using both parameters.

The application of the pattern MultipleValuedERefToSet is shown in Fig. 30 where the first parameter is the EReference expression for selectedFloors. It is a multi-valued reference from linked_lift to linked_floors. The second parameter is the set call_buttons.

6.3.3 Enum type to Boolean

Pattern EnumTypeToBoolean is shown in Fig. 31 where it is defined to accept four parameters. This pattern maps an enumeration type attribute (with two values) to a Boolean. The first parameter of this pattern is the attribute expression, the second one is the Boolean expression. The other two parameters: aVal1 and aVal2 are the values of enumeration type attribute. The pattern produces two invariants, each with a different value of enumeration type attribute. One value is mapped to the FALSE, and the other one is mapped to the TRUE value of Boolean.

```
pattern EnumTypeToBoolean
(attributeExp bExp aVal1 aVal2)
alias attributeExp : attributeExp
alias bExp : bExp
alias aVal1 : aVal1
alias aVal2 : aVal2
{
INVARIANT :
"("attributeExp"="aVal1"<=>"bExp"=""FALSE"")"
"&"
"("attributeExp"="aVal2"<=>"bExp"=""TRUE"")"
}
```

Fig. 31 EnumTypeToBoolean Pattern

```
applyPattern Lift.EnumTypeToBoolean(
    B{Door(linked_lift)}B
    door_open
    Closed
    Open
)
applyPattern Lift.EnumTypeToBoolean(
    B{Direction(linked_lift)}B
    direction_up
    Down
    Up
)
```

Fig. 32 EnumTypeToBoolean Application

Figure 32 presents the application of EnumTypeToBoolean pattern with two applyPatterns. This pattern accepts four parameters. In the first applyPattern, the first parameter is the expression for the enumeration type attribute Door of EClass linked_lift and it is mapped to the Boolean door_open which can be expressed in the second parameter as a B expression. The third parameter aVal1 is the value Closed of attribute Door and similarly, the fourth parameter aVal2 is the value Open of attribute Door.

In the second applyPattern, the enumeration type attribute Direction of EClass linked_lift is mapped to the Boolean door_open. Parameters aVal1 and aVal2 are the enumeration values: Down and Up respectively.

The result of applying the pattern EnumTypeToBoolean can be seen in Sect. 5 Listing 5.

We also defined two BClauses in the application for the complete generation of lift linkage B specification. First one is for the initialisation and the other one is for the operations.

7 Other applications

We applied our approach to several other examples, such as the *Scheduler* example discussed in [17, 21] and the ERTMS/ETCS railway system, which is a more realistic case study [22]. As the objective is to visualize a formal specification using a DSL in order to understand it, we do not present the formal model in this section. Rather, we provide screenshots and identify some interesting notions that one can exhibit without looking at the formal specification.

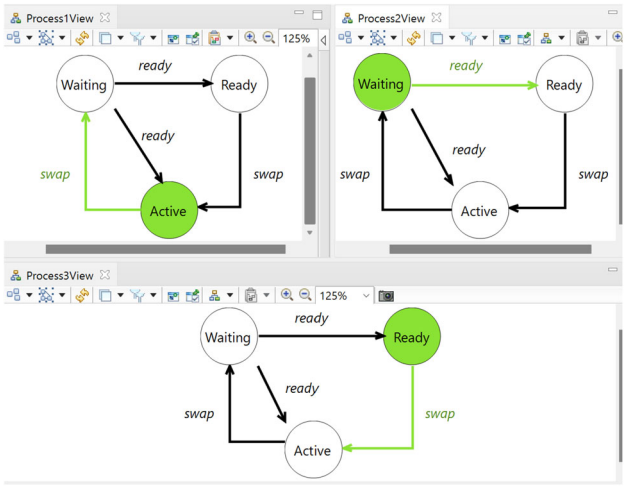


Fig. 33 Graphical animation of the Scheduler example

7.1 Scheduler

Figure 33 applies state-transition diagrams to visualize the behaviour of processes managed by the Scheduler example. This model introduces three processes (one state-transition diagram per process) and deals with three states: Waiting, Ready, and Active. Transitions refer to the various operations of the formal specification, which are: new, ready, del (delete), and swap. Transitions ready and swap may involve several processes simultaneously. The highlighted transition shows the next enabled operations, and the highlighted state shows the current state of the process.

Process1 showed in *Process1View* is in state Active whereas the animatable operation from this state is swap. *Process2View* and *Process3View* illustrate Process2 and Process3 whose current states are respectively Waiting and Ready. These state/transition diagrams emphasize on some properties such as deadlock freedom, or the non-blocking property. Indeed, one can see that after being active a process does not stop the system. In fact, the animation of operation swap puts Process1 in state waiting and activates one ready process. In this case, it activates Process3.

Using Meeduse, we generated the Static B specification of the scheduler DSL. The generated specification includes three Sets and six variables (alongside their invariants and empty initialization), and operations (setters, getters, etc). This scheduler example helped us to explore a new mapping rule. In existing scheduler B specification, we have 3 variables: active, ready, waiting which are defined as sets of processes. We defined these concepts in the DSL via attribute Status, whose type is enumeration StatusEnum. Mapping

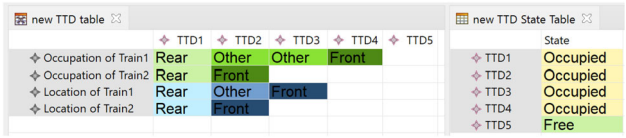


Fig. 34 Graphical animation of the ERTMS/ETCS Case Study

enumerations to sets introduces the following invariants in the linkage machine:

Listing 12: EnumType to a Set

INVARIANT
$linked\_process[\mathbf{dom}(Status \rhd \{Waiting\})] = waiting$
$\wedge\ linked\_process[\mathbf{dom}(Status \rhd \{Ready\})] = ready$
$\wedge\ linked\_process[\mathbf{dom}(Status \rhd \{Active\})] = active$

7.2 ERTMS/ETCS case study

Lift and Scheduler are used as proofs of concepts, which show the applicability of our approach. We also applied the approach to a realistic example of the ERTMS/ETCS railway system. ERTMS stands for European Rail Traffic Management System, and ETCS stands for European Train Control System. It is a set of specifications introduced as a standard for a common interoperable platform for railway management and signaling systems. It is classified into different functional levels according to the equipment used and operating mode. The levels are ETCS level 1, level 2, and level 3. The case study used in this work is based on the formal B specification of [22]. The latter is published in the ABZ 2018 [23] case study, which is related to the Hybrid ERTMS/ETCS [24] Level 3 standard. In level 3, a given train continuously computes its position using a GNSS[4] device (satellite-based localization system).

The specification used in the current case study is composed of four components (one B machine and three refinements). This allowed us to evaluate the feasibility and scalability of our technique and to explore various other mapping rules. Figure 34 shows the underlying graphical animation. The current position of a train (called occupation) and the known position (called location) are illustrated via a table. The concept of TTD (Trackside Train Detection) refers to a railway section. For example, Train1 occupies four sections (the rear is on TTD1 and the front is on TTD4), but the known position of the head of Train1 is TTD3. The right-hand side of the figure shows the state of each TTD; it can be either occupied or free. From this view, one can understand that the specification does not (yet) prevent accidents, since sections can be occupied by several trains.

[4] Global Navigation Satellite System. https://www.euspa.europa.eu/european-space/eu-space-programme/what-gnss.

```
template scheduler
pattern EClassToExtendedConstant
(anEClass aConstant userDefined)
alias anEClass:anEClass
alias aConstant: aConstant
alias consName: "Linked_"anEClass
alias userDefined : userDefined
alias relation : anEClass ">->>" aConstant
{
    CONSTANTS:
        consName
    PROPERTIES:
        consName ":" relation
        " & " userDefined
}
pattern ReferenceToVariable(bExp aVar)
alias bExp: bExp
alias aVar: aVar
{
    INVARIANT:
        aVar "=" bExp
}
```

Fig. 35 Scheduler Pattern Template

7.3 Patterns for the scheduler

In the previous sub-section we showed the linkage B specification that we defined to the Scheduler example. To automatically produce this specification, we defined patterns in a template called scheduler that is shown in Fig. 35. The template consists of two patterns: EClassToExtendedConstant and ReferenceToVariable. Pattern EClassToExtendedConstant is defined with three parameters (anEClass, aConstant and userDefined) and maps an EClass to a constant. The pattern contains four aliases, and it will produce a constant and a property. In clause CONSTANT, this pattern adds a constant name, represented with alias consName whose value is the concatenation of string "Linked_" with the value of parameter anEClass. Alias relation is defined as a bijection from parameter anEClass to parameter aConstant. Pattern ReferenceToVariable maps a reference to a variable and it is defined with two parameters: bExp (B expression) and aVar (Variable). It only produces an invariant. Next in this subsection, we discuss the application of both patterns.

7.3.1 EClass to extended constant

The application of pattern EClassToExtendedConstant is demonstrated in Fig. 36 where it takes the class PROCESSCLASS as the first parameter anEClass. The second parameter is a constant PID. The third parameter is userdefined which is defined as the cardinality of PROCESSCLASS

```
applyPattern scheduler.EClassToExtendedConstant(
    PROCESSCLASS
    PID
    B{ card(PROCESSCLASS) = card(PID) }B
)
```

Fig. 36 EClassToExtendedConstant Application

```
applyPattern scheduler.ReferenceToVariable(
    B{linked_process[dom(Status|>{Waiting}) ]}B
    waiting
)
applyPattern scheduler.ReferenceToVariable(
    B{linked_process[dom(Status|>{Ready}) ]}B
    ready
)
applyPattern scheduler.ReferenceToVariable(
    B{linked_process[dom(Status|>{Active}) ]}B
    active
)
```

Fig. 37 ReferenceToVariable Application

equal to the cardinality of PID (card(PROCESSCLASS)= card(PID)).

The result of applying the pattern EClassToExtendedConstant can be seen in Listing 13 below.

Listing 13: EClass to extended Constant

CONSTANTS
Linked_PROCESSCLASS
PROPERTIES
Linked_PROCESSCLASS $\in$ *PROCESSCLASS* $\rightarrowtail\!\!\!\!\!\rightarrow$
PID
$\wedge$ **card**(*PROCESSCLASS*) = **card**(*PID*)

7.3.2 Reference to variable

Pattern ReferenceToVariable is applied three times in the application as shown in Fig. 37. The first parameter contains the reference as a B expression. In these applyPatterns, the variables waiting, ready, and active from existing scheduler B specification are used in the second parameter. They are mapped with their corresponding equivalent concepts (Waiting, Ready, and Active) from the DSL machine.

In this example of the scheduler, we used the Refinement/Inclusion technique. To generate the complete scheduler B specification, we used the BClause tag three times in the application for definition, initialization, and operations.

7.4 Patterns for ERTMS/ETCS

To use our tool on the realistic case study of ERTMS/ETCS, we defined eight patterns in the Pattern-Definition DSL. The patterns are defined based on the ERTMS/ETCS B specification provided by Amel Mammar [22]. In our previous examples, we already illustrated the application of four out of the eight patterns. The already illustrated patterns are EClassToExtendedConstant, EclassToBMachine, EnumTypeToBoolean and ReferenceToVariable. In the following, we show four new patterns.

```
pattern EClassToConstant
(anEClass aConstant)
alias anEClass:anEClass
alias aConstant: aConstant
alias consName: "Linked_"anEClass
alias relation : anEClass ">->>" aConstant
{
    CONSTANTS:
        consName
    PROPERTIES:
        consName ":" relation
}
```

Fig. 38 EClassToConstant Pattern

```
applyPattern amel.EClassToConstant(
    Train
    Trains)
```

Fig. 39 EClassToConstant Application

7.4.1 EClass to constant

Pattern EClassToConstant is shown in Fig. 38. It is defined to map an EClass to a constant. The difference between the pattern EClassToConstant and the pattern EClassToExtendedConstant is that the former takes two parameters: anEClass and aConstant. Whereas EClassToExtendedConstant takes three parameters, including the userDefined which was used to produce an additional invariant. Apart from the parameter userDefined, pattern EClassToConstant is the same as EClassToExtendedConstant.

Figure 39 gives the application of EClassToConstant pattern. The first parameter is EClass Train from DSL, and the second parameter is the constant Trains from ETCS specification.

7.4.2 Enumeration to set

To map an enumeration to a set, we defined a pattern EnumToSet with two parameters which can be seen in Fig. 40. It is defined with two parameters: anEnum (Enumerator) and bExp (B expression). It will produce a constant and a property.

The application of Pattern EnumToSet is shown in Fig. 41. The first parameter is enumeration Status from the DSL and the second parameter is a B expression. In B expression, the elements of set stateTTD from ETCS machine are linked to the corresponding values of enumeration Status.

```
pattern EnumToSet(anEnum bExp)
alias consName: "Linked_"anEnum
alias bExp: bExp
{
    CONSTANTS:
        consName
    PROPERTIES:
        consName "={" bExp "}"
}
```

Fig. 40 EnumToSet Pattern

```
applyPattern amel.EnumToSet
(
Status
B{freeT|->Free,occupiedT|->Occupied}B
)
```

Fig. 41 EnumToSet Application

```
pattern BoolAttributeToBoolVariable
(consVarName anAttribute aVar)
alias consVarName : consVarName
alias anAttribute: anAttribute
alias aVar: aVar
{
VARIABLES: aVar
INVARIANT: anAttribute"=("consVarName","aVar")"
}
```

Fig. 42 BoolAttributeToBoolVariable Pattern

```
applyPattern amel.BoolAttributeToBoolVariable
(
    Linked_Train
    Connected
    isConnected
)
```

Fig. 43 BoolAttributeToBoolVariable Application

7.4.3 Boolean attribute to boolean variable

Figure 42 gives the pattern to map a Boolean attribute to a Boolean variable. The pattern is defined with three parameters. The first parameter is called consVarName; it can be either a constant or a variable. The second parameter is an attribute from DSL, and the third one is a variable from the existing B machine. The pattern produces the variable of the existing machine in the linkage specification. It also produces an invariant.

Pattern BoolAttributeToBoolVariable is applied to map the attribute Connected (second parameter) from DSL, and the variable isConnected (third parameter) from the ETCS machine, which can be seen in Fig. 43. The first parameter consVarName is constant Linked_Train.

```
pattern EnumTypeAttributeToSetValuedVariable
(anAttribute consVarName1 aVar consVarName2)
alias consVarName1 : consVarName1
alias consVarName2 : consVarName2
alias anAttribute: anAttribute
alias aVar: aVar
{
VARIABLES: aVar
INVARIANT:
anAttribute"=("consVarName1","
             aVar","consVarName2")"
}
```

Fig. 44 EnumTypeAttributeToSetValuedVariable Pattern

```
applyPattern
amel.EnumTypeAttributeToSetValuedVariable
(
    TrackStatus
    Linked_Trackside
    stateTTD
    Linked_Status
)
```

Fig. 45 EnumTypeAttributeToSetValuedVariable Application

7.4.4 Enum type attribute to set valued variable

The pattern to map an enumeration type attribute with a variable containing set values is given in Fig. 44. The pattern is defined with four parameters. The first parameter is the enumeration type attribute, and the second parameter is consVarName1, which is a constant or variable related to the attribute. The third parameter is the variable from the existing machine, and the last parameter is the consVarName2, which is related to the variable. The pattern also produces the variable of the existing machine along with an invariant in the linkage specification.

The application of the pattern EnumTypeAttributeToSetValuedVariable can be seen in Fig. 45. It maps the attribute TrackStatus from DSL to the variable stateTTD of the existing ETCS machine. The second parameter consVarName1 is Linked_Trackside, and the fourth parameter consVarName2 is Linked_Status.

Note that the ERTMS/ETCS B specification comprises one machine (M0) and three refinements (M1, M2, M3). Using the above patterns from the Pattern-Definition DSL, we applied the patterns in four separate application files of the Pattern-Application DSL, which produced four different linkage machines. The number of applyPatterns in each file varies, as each machine in the ERTMS/ETCS specification varies according to the number of constants, properties, variables, and invariants. The applyPatterns in M0, M1, M2, and M3 are 10, 12, 14, and 26, respectively. In each application file, two BClauses are defined: one for initialization and another for operations.

8 Discussion

Sections 5 and 6 presented the general concepts of our approach and illustrated them using a simple example, that of a Lift. Section 7 addressed other examples showing the flexibility and scalability of the approach. We are aware that additional efforts are required to prove scalability, but we believe that the obtained results are promising: first, we were able to deal with complex specifications, such as that of ERTMS/ETCS with hundreds of lines of B code; second, we addressed three models with different kinds of data structures; and last, we showed that our patterns can be reused among these examples.

To show the complexity of the various specifications and the corresponding linkage specifications, we provide Table 1. The comparison is based on produced linkage constants, properties, variables, and invariants. The linkage constants are produced by linking the constants and sets. Linkage properties are produced to define these constants or to refine existing constants. We only produced linkage variables during the Lift example; in other examples, the mapping of existing and DSL variables is done through linkage invariants. Table 1 also shows that the ERTMS example is two to ten times bigger (in terms of lines) than the small examples of Lift and Scheduler mentioned in this paper.

Tables 1 and 2 partially prove how scalable is our approach, regarding the size of the various B specifications, and also the reusability of our patterns. Every B specifications has its own characteristics. The Lift deals with simple data types (mainly integer and boolean), whereas the Scheduler is built on abstract data structures (such as abstract sets). The ERTMS/ETCS specification gathers both kinds of data structures and brings the notion of refinement. The heterogeneity of these specifications and the underlying patterns presented in this paper, allowed us to present an exhaustive work. This is especially helpful when adopting our approach for specification validation. As some aspects of our approach are automated, such as generating the initial static B semantics from the DSL meta-model and the mappings. The complexity and effort required increase with the system specifications, model sizes, meta-models, and the scale of the DSL. While automation assists with initial steps, more detailed and intricate models demand significant manual effort to ensure comprehensive linkage.

The particularity of our approach in the context of visual animation is that it applies DSLs. Hence, the objective is not to enhance the performance of visual animation or DSLs but to make visual animation more pragmatic. In fact, in our approach, the graphical input models that visualize the behavior of the specification are provided by the end-user herself, which is not the case in most existing visual animation tools. To demonstrate how using DSLs enhances usability, we presented the animation of our ERTMS/ETCS models to a railway expert who identified three unexpected behaviors and concluded that "*The DSL is run using various scenarios generating behaviors that often surprise a railway expert. The simulation analysis by an expert helps to define the semantic limits of a given DSL but the more interesting part comes when a misunderstanding of the specification is identified. Even authors of the code never identified the problem before. It shows that basic specification errors happens and are really difficult to detect without a graphical animation.*". We acknowledge that while our results provided in this paper are promising, but conducting other double-blind randomized trials would provide more robust evidence and enhance confidence in our these preliminary findings. And We have perspectives to address this in our future research.

Table 1 Comparison of Linkage Specifications

	Lift	Scheduler	ERTMS/ETCS			
			M0	M1	M2	M3
Linkage Constants		1	5	5	6	7
Linkage Properties	2	2	6	6	9	10
Linkage Variables	2					
Linkage Invariants	9	3	6	8	8	11
Size (lines) of Linkage Specification	117	105	247	341	415	1000

Table 2 Comparison of Defined and Applied Patterns

Example	Defined Patterns	Applied Patterns	B Clauses
Lift	5	6	2
Scheduler	2	4	3
ERTMS M0	8	10	2
ERTMS M1	8	12	2
ERTMS M2	8	14	2
ERTMS M3	8	26	2

9 Related work

Mixing formal methods and domain-specific languages has been addressed in several works [25–28], but often with the intention of providing formal semantics to a DSL and hence being able to reason about its correctness. In fact, the strength of formal methods originates from their precision and the availability of automated reasoning tools. An exhaustive state of the art about this topic is provided in [14]. The work presented in this paper addresses a reverse approach to use a

DSL as a way to remedy the poor readability of a formal specification since the strength of DSLs is the domain-centric notations. Zalila et al., in [27] suggest the use of an additional DSL to feedback formal verification results. The authors propose a new language (called FEVEREL) that allows the designer to implement the verification result feedback from the formal level to the DSL level. The approach is close to visual animation, but the use of additional languages may weaken the acceptance of the underlying technique because it requires new skills. The work of Tikhonova [29] starts from a DSL meta-model, generates Event-B specifications, but applies classical visual animation using BMotion Studio to the formal specifications in order to understand the behaviour of the DSL. The limitation is that the DSL syntax must be reworked in BMotion Studio, which requires additional verifications to check the compatibility between the initial DSL syntax and the graphical visualization.

Our proposal is to use the DSL itself in order to ensure visual animation. As far as we know, this technique has not been investigated before, mainly because, apart from Meeduse, none of the existing language workbenches provides execution features that are built on a well-established formal method such as B. This claim is attested by the survey of Iung et al., [30] in which the formal dimension is completely absent. This survey shows that among the 59 discussed language workbenches, only 9 provide support for verification, which is only done by testing. The interesting aspect of Meeduse in the current work is that it builds on the B method and applies ProB to execute the DSL. The approach does not need any additional implementation effort. Once the DSL is done, one can reuse a formal specification that is provided by a formal methods expert to execute it.

It is known that visual animation is highly desirable when applying formal methods tools because the poor readability of formal notations prevents user involvement during the validation activity. The approach of this paper complements the numerous tools and efforts dedicated to the B method: BRAMA [8], AnimB[5] B-Motion Studio [5], B-Motion Web [9], and VisB [10]. The added value with regards to these tools is that the B method expert only applies B to link his/her specifications to the formal static semantics of the DSL. In [17], Leuschel et al., propose a simple way of producing custom animations. The idea is to assemble a series of pictures and write an animation function in B itself, which stipulates which pictures should be shown where depending on the current state of the system. Our approach builds on a similar argument but starts from graphical models that are designed by domain experts using a DSL tool crafted by an MDE expert.

In this work, we mainly addressed animation in B and used ProB tool support. Nonetheless, most formal tools provide animation techniques to deal with validation. Liu et al., in [31] report about their experience in applying formal methods in industry, in Japan and China, and mention that "most users of software systems developed in industry would not be expected to understand formal specifications," and therefore, the formal specification written by the developer needs to be validated against the user's requirements in a comprehensible manner. Currently, specification animation is the most used technique [32, 33]. However, one major difficulty of specification animation is that it requires that humans understand what the animation is demonstrating. This explains why many animation tools provide visual animation (also called graphical animation). Unfortunately, classical visual animation builds on graphical representations that are provided by the developer, not by the end-user. More recent approaches, such as [6], propose to generate comprehensible and representative animation data based on the specification to visually demonstrate the relationships between input and output values and the characteristics of the values for the user who is usually interested in the behaviour of the system under construction. As far as we know, none of the existing animation tools explored DSLs like the proposal of this paper. But exploring their technologies may enhance the current work. For example, we may try to generate animation data, being inspired by [6], and use these data as DSL inputs, since the DSL is expected to be used by end-users themselves.

10 Conclusion

This paper presented an approach for the visual animation of B specifications, using an MDE paradigm built on DSLs. Our objective was to ensure validation thanks to domain-centric notations that are expected to be more comprehensible for domain experts than the formal specifications. We used Meeduse, a language workbench dedicated to formally instrument the static and dynamic semantics of a DSL by means of B models.

We provided a linkage B specification that maps the B models managed by Meeduse and the B specification to be visualized. Two strategies are provided and discussed in this paper: (*i*) Refinement/Inclusion, and (*ii*) Inclusion/Inclusion. The examples mentioned in this paper are covered to show the viability of our technique. Writing the linkage B specification is easier when the DSL is based on the same concepts as the existing B specification. DSLs can also be used to visualize specifications that are defined with several refinement levels. For space reasons, we did not develop this aspect here.

Besides, we showed that our approach also favors the reuse of existing specifications like the dynamic semantics of a DSL. It can be done with minor modifications of the existing specification (e.g., by adding operations such as setValues and update_existing). During our previous work

[5] AnimB: https://wiki.event-b.org/index.php/AnimB,

[16], the linkage B specifications were written manually but systematically. In this article, we update the previous work with a DSL tool to generate linkage specifications (semi)-automatically. We developed two DSLs: Pattern-Definition and Pattern-Application. Pattern-Definition allows the user to define patterns for B predicates. Pattern-Application is used to apply the defined patterns (of the Pattern-Definition DSL) alongside structuring the body of the linkage B structure. We experimented with our pattern-based approach using examples of Lift, Scheduler, and ERTMS/ETCS. We expect that it simplifies the work of B method experts, and it will soon be integrated into Meeduse.

Still, this work has some limitations. First, we have assumed that the DSL and the B model are independent of each other so that the DSL may provide useful insights to the end-user. After doing our experiments, we believe that a DSL meta-model may be extracted from the B specification. This point belongs to our perspectives: the idea is to analyze the data structures of the B specifications and automatically identify a useful meta-model so that the MDE expert could associate a relevant graphical syntax. Second, refinements were not fully investigated. In our current work, the refinement has been addressed via the ERTMS/ETCS case study. We think that we may provide a generic technique that builds the DSL (and hence the visual animation) incrementally, being inspired by B refinements.

The framework provided in this paper is primarily designed for the B Method but it could be adaptable to other formal languages. This adaptation involves developing static and dynamic semantics for the new language, creating linkage specifications, and integrating tools like ProB to execute and visualize the new specifications. The effort required depends on the language's complexity, tool compatibility, available expertise, and the scope of adaptation.

Availability of B Specifications. All the artifacts of this paper (including B specifications, DSLs, and linkage machines) are publicly available in the Meeduse GitHub repository: Lift,[6] Scheduler,[7] and ETCS[8] including our Linkage Generator Tool.[9]

Author Contributions All the authors contributed equally to this work.

Declarations

Conflict of interest The authors declare no Conflict of interest.

[6] https://github.com/meeduse/Samples/tree/main/Lift.

[7] https://github.com/meeduse/Samples/tree/main/Scheduler.

[8] https://github.com/meeduse/Samples/tree/main/ETCSLevel3.

[9] https://github.com/meeduse/Samples/tree/main/LinkageGeneratorTool.

References

1. Bowen JP, Hinchey MG (1995) Seven more myths of formal methods. IEEE Softw 12:34–41
2. Gaudel MC (1991) Advantages and limits of formal approaches for ultra-high dependability. In *Proceedings of the 6th international workshop on software specification and design*, IWSSD '91, 237–241 (IEEE Computer Society Press, Washington, DC, USA,)
3. Abrial J-R (1996) The B-Book: Assigning Programs to Meanings. Cambridge University Press, USA
4. Andova S, van den Brand M, Engelen L, Verhoeff T (2012) MDE basics with a DSL focus. In *Formal methods for model-driven engineering - 12th international school on formal methods for the design of computer, communication, and software systems*, vol. 7320 of *LNCS*, 21–57 (Springer,)
5. Ladenberger L, Bendisposto J, Leuschel M (2009) Visualising Event-B models with B-Motion Studio. In: Alpuente M, Cook B, Joubert C (eds) Formal Methods for Industrial Critical Systems, 202–204 (Springer. Berlin Heidelberg, Berlin, Heidelberg)
6. Liu S, Miao W (2021) A formal specification animation method for operation validation. J Syst Softw 178:110948
7. Watson N, Reeves S, Masci P (2018) Integrating user design and formal models within PVSio-Web. Electr Proc Theoretical Comput Sci **284**
8. Servat T (2006) Brama: A new graphic animation tool for B models. In: Julliand J, Kouchnarenko O (eds) B 2007: Formal Specification and Development in B, 274–276 (Springer. Berlin Heidelberg, Berlin, Heidelberg)
9. Ladenberger L, Leuschel M (2016) BMotionWeb: A tool for rapid creation of formal prototypes. In De Nicola, R. & Kühn, E. (eds.) *Software Engineering and Formal Methods*, 403–417 (Springer International Publishing, Cham,)
10. Werth M, Leuschel M (2020) VisB: A lightweight tool to visualize formal models with SVG graphics. In Raschke, A., Méry, D. & Houdek, F. (eds.) *Rigorous State-Based Methods*, 260–265 (Springer International Publishing, Cham,)
11. Krings S, Körner P (2021) Prototyping games using formal methods. In Cerone, A. & Roggenbach, M. (eds.) *Formal Methods – Fun for Everybody*, 124–142 (Springer International Publishing, Cham,)
12. Steinberg D, Budinsky F, Paternostro M, Merks E (2009) *EMF: Eclipse Modeling Frameworke* (Addison-Wesley,), 2nd edn
13. Idani A (2020) Meeduse: A tool to build and run proved DSLs. In Dongol, B. & Troubitsyna, E. (eds.) *Integrated Formal Methods*, 349–367 (Springer International Publishing, Cham,)
14. Idani A, Ledru Y, Vega G (2020) Alliance of model-driven engineering with a proof-based formal approach. Innov Syst Softw Eng 16:289–307
15. Leuschel M, Butler M (2003) Prob: A model checker for B. In: Araki K, Gnesi S, Mandrioli D (eds) FME 2003: Formal Methods, 855–874 (Springer. Berlin Heidelberg, Berlin, Heidelberg)
16. Yar A, Idani A, Ledru Y, Collart-Dutilleul S (2022) Visual animation of b specifications using executable dsls. In *Proceedings of the 25th international conference on model driven engineering languages and systems: companion proceedings*, MODELS '22, 617–626 (Association for Computing Machinery, New York, NY, USA,). https://dl.acm.org/doi/10.1145/3550356.3561585
17. Leuschel M, Samia M, Bendisposto J (2008) Easy graphical animation and formula visualisation for teaching B
18. Müllerburg M, Holenderski L, Maffeïs O, Merceron A, Morley M (1995) Systematic testing and formal verification to validate reactive programs. Softw Qual J 4:287–307. https://doi.org/10.1007/BF00402649
19. Eclipse sirius (2022). https://www.eclipse.org/sirius/
20. Acceleo. https://www.eclipse.org/acceleo/. Accessed: 2023-02-05

21. Legeard B, Peureux F, Utting M (2002) Automated boundary testing from Z and B. In: Eriksson L-H, Lindsay PA (eds) FME 2002: Formal Methods-Getting IT Right, 21–40. Springer, Berlin Heidelberg, Berlin, Heidelberg
22. Mammar A, Frappier M, Fotso SJT, Laleau R (2020) A formal refinement-based analysis of the hybrid ERTMS/ETCS level 3 standard. Int J Softw Tools Technol Transfer 22:333–347
23. Butler MJ, Raschke A, Hoang TS, Reichl K (2018) (eds.). *6th International Conference, ABZ*, vol. 10817 of *LNCS* (Springer,)
24. The ERTMS/ETCS signalling system. http://www.railwaysignalling.eu/wp-content/uploads/2016/09/ERTMS_ETCS_signalling_system_revF.pdf. Accessed: 2022-03-16
25. Bodeveix J-P, Filali M, Lawall J, Muller G (2005) Formal methods meet domain specific languages. In *Proceedings of the 5th international conference on integrated formal methods*, 187–206 (Springer,)
26. Rivera J, Durán F, Vallecillo A (2009) Formal specification and analysis of domain specific models using Maude. Simulation 85:778–792
27. Zalila F, Crégut X, Pantel M (2016) A DSL to Feedback Formal Verification Results. In Famelis, M., Ratiu, D. & Selim, G. M. K. (eds.) *13th Model-Driven Engineering, Verification and Validation Workshop at MODELS conference 2016 (MoDeVVa 2016)*, vol. 1713, 30–39 (CEUR-WS : Workshop proceedings, Saint Malo, France,). https://hal.archives-ouvertes.fr/hal-03172263
28. Meyers B, Vangheluwe H, Denil J, Salay R (2020) A framework for temporal verification support in domain-specific modelling. IEEE Trans Software Eng 46:362–404
29. Tikhonova U, Manders M, Brand van den M, Andova S, Verhoeff T (2013) Applying model transformation and Event-B for specifying an industrial DSL. In *Workshop on Model Driven Engineering, Verification and Validation*, CEUR Workshop Proceedings, 41–50 (CEUR-WS.org,)
30. Iung A et al (2020) Systematic mapping study on domain-specific language development tools. Empir Softw Eng 25:4205–4249
31. Liu S, Takahashi K, Hayashi T, Nakayama T (2009) Teaching formal methods in the context of software engineering. ACM SIGCSE Bulletin 41:17–23
32. Hazel D, Strooper P, Traynor O (1997) Possum: an animator for the sum specification language. In *Proceedings of Joint 4th international computer science conference and 4th Asia Pacific software engineering conference*, 42–51
33. Miller T, Strooper P (2003) A framework and tool support for the systematic testing of model-based specifications. ACM Trans Softw Eng Methodol 12:409–439. https://doi.org/10.1145/990010.990012